Author Validation Tasks: Executive Roadmap & Practical Guide

For Authors: Md. Ashikul Islam & Shahriar Hasan

Manuscript: *Maintainer Prioritization in npm Supply Chains: Objectives, Representation Sensitivity, and Metadata Limits*

Target Journal: Empirical Software Engineering (EMSE), Springer

Executive Summary at a Glance

This separate executive document explains **what** the author validation package is about, **what tasks** you must complete, **which direction** to pursue, and **how to handle** each task efficiently and rigorously.

- **Task 1 (Human Audit):** Each author independently audits 120 sampled repositories across 5 static evidence criteria (600 decisions each, 1,200 total), followed by joint adjudication of disagreements.
- **Task 2 (Author Contributions):** Document each author's real, factual contributions using the CRediT role matrix.
- **Task 3 (Manuscript Approval):** Both authors independently read, verify, and formally approve the 34-page final EMSE submission PDF.
- **Integration & Handoff:** Run the validation pipeline scripts, compute Cohen's κ, update manuscript tables, and submit corrections to Editorial Manager.

Prepared: 30 October 2026
Workplace Reference: `/home/ashik/Desktop/npm-supply-chain-research/tasks/author_validation_2026-10-28/`
Status: Clear Action Plan & Procedural Handbook

Contents

1 What This Package Is About (Context & Purpose) 2

1.1 Research Context . . . 2

1.2 Why Author Human Validation Is Needed . . . 2

1.3 Three Inviolable Principles . . . 2

2 What Type of Tasks You Need to Complete 2

2.1 Task 1: Independent 120-Endpoint Human Audit (600 Decisions Each) . . . 2

2.2 Task 2: Author Contributions Statement . . . 3

2.3 Task 3: Final Manuscript Review and Approval . . . 3

3 In Which Direction You Should Go (Execution Roadmap) 5

3.1 Phase 0: Calibration and Setup (Day 1) . . . 5

3.2 Phase 1: Independent Review Passes (Days 2 to 10) . . . 5

3.3 Phase 2: Reconciliation and Adjudication (Day 11) . . . 5

3.4 Phase 3: Formal Documentation (Day 12) . . . 6

3.5 Phase 4: Pipeline Integration & Journal Submission (Day 13) . . . 6

4 How to Handle These Tasks Practically (Inspection Protocol) 7

4.1 Standard Operating Procedure for Every Endpoint (8 Steps) . . . 7

4.2 Decision Matrix: Supported vs Unsupported vs Unknown . . . 7

5 Key Edge Cases and Practical Tips 9

5.1 How to Handle Monorepos (e.g. A002: arthurfiorette/tinylibs) . . . 9

5.2 Handling Missing Data or 404s . . . 9

5.3 Critical Do's and Don'ts . . . 9

6 Master Reference of Files and Locations 9

1 What This Package Is About (Context & Purpose)

1.1 Research Context

In our research study submitted to *Empirical Software Engineering* (EMSE), we investigate how maintainer prioritization algorithms in npm are sensitive to objective scalarizations, graph representations, and metadata availability limits.

As part of the comprehensive revision and journal validation (Reports 22–24):

1. All automated algorithmic benchmarks, 288 method comparisons, and 100-root prospective fresh cohort runs have been fully executed and verified.
2. A probability sample of **120 GitHub repository endpoints** was frozen across 4 source-cardinality strata (30 per stratum) from 7,158 source-reachable endpoints.
3. Machine-automated collectors extracted provisional indicators (such as whether workflows or lockfiles exist).

1.2 Why Author Human Validation Is Needed

In the current manuscript, we explicitly report automated indicators while disclosing that **they have not yet been manually validated by humans**.

Reviewers in top empirical software engineering journals expect evidence that automated heuristics accurately reflect ground truth. By conducting an independent human audit:

- You provide **empirical ground truth** verifying whether npm packages legitimately map to pinned repository workflows and lockfiles.
- You calculate formal **inter-rater reliability** (Cohen's κ and percentage agreement) before consensus adjudication.
- You transform automated indicators into **audited scientific findings**, substantially increasing the rigor and defensibility of the paper.

1.3 Three Inviolable Principles

1. **Strict Independence:** Neither author may see, discuss, or copy the other's initial label decisions during the initial pass.
2. **Auditable Evidence:** Every label must be backed by concrete file paths and code references in the evidence notes.
3. **Legitimate Uncertainty:** If evidence is ambiguous, missing, or inaccessible, the valid label is `unknown`. Never guess or force a `supported` label.

2 What Type of Tasks You Need to Complete

There are **three core tasks** required from the human authors, followed by the computational handoff:

2.1 Task 1: Independent 120-Endpoint Human Audit (600 Decisions Each)

Each author opens their assigned, dedicated CSV spreadsheet in `tasks/author_validation_2026-10-28/reviews/`:

- `Md_Ashikul_Islam_independent_labels.csv`
- `Shahriar_Hasan_independent_labels.csv`

Both sheets contain the identical 120 audit IDs (`A001` to `A120`). For each endpoint, you inspect the static repository evidence at the exact pinned commit SHA and record 5 distinct labels:

1. **Criterion 1: `repository_mapping_supported`**
   - *Question:* Does the npm registry repository field accurately link to this GitHub repository, and does the relevant `package.json` (at the root or in a monorepo workspace) declare this package name?
2. **Criterion 2: `workflow_file_present`**
   - *Question:* Is there an actual GitHub Actions workflow YAML file in `.github/workflows/` at the pinned commit?
3. **Criterion 3: `install_directive_observed`**
   - *Question:* Do workflow steps contain an active shell instruction (e.g. `npmci`, `npminstall`, `pnpminstall`, `yarninstall`) that applies to this package or its root workspace?
4. **Criterion 4: `lockfile_present`**
   - *Question:* Is there a relevant lockfile (`package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`, `npm-shrinkwrap.json`) governing the package?
5. **Criterion 5: `dependency_version_supported`**
   - *Question:* Can you find at least one concrete dependency declared in `package.json` and locate its corresponding resolved, pinned version in the applicable lockfile?

Allowed Label Values: Each cell must contain *only* one of these three lowercase strings:

- `supported` — The static evidence clearly satisfies the criterion.
- `unsupported` — The evidence affirmatively contradicts the criterion or complete tree inspection proves its absence.
- `unknown` — The evidence is ambiguous, incomplete, or acquisition failed (e.g. 404, rate limit, capped candidate tree).

2.2 Task 2: Author Contributions Statement

Open `tasks/author_validation_2026-10-28/templates/actual_contributions.csv`.

- Record short, factual statements describing what each author actually did across CRediT roles (Conceptualization, Methodology, Software, Data Curation, Validation, Analysis, Writing).
- Roles must reflect real historical and current work, not assigned merely based on first vs second author order.

2.3 Task 3: Final Manuscript Review and Approval

Open `tasks/author_validation_2026-10-28/templates/manuscript_approval.md`.

- Both authors must read the complete 34-page final EMSE article PDF (not just this guide).
- Verify author spelling (**Md. Ashikul Islam** and **Shahriar Hasan**), affiliations, email contacts, and ORCIDs.

- Check consistency between figures, tables, and narrative claims.
- Sign off on the approval checklist with the exact PDF filename, date, and SHA-256 hash.

3 In Which Direction You Should Go (Execution Roadmap)

To complete these tasks smoothly without burnout or protocol deviations, follow this 5-phase sequential roadmap:

Phase	Objective	Key Actions & Deliverables	Time
Phase 0	Calibration & Setup	Open `Evidence_Index.html`; do calibration on 5 real rows.	1–2 hrs
Phase 1	Independent Review Passes	Each author completes 120 rows in their own CSV across 12 batches. Run `validate_reviews.py`.	15–25 hrs / author
Phase 2	Preservation & Adjudication	Freeze initial CSVs (SHA-256). Run adjudication generator. Resolve in `adjudication.csv`.	2–4 hrs
Phase 3	Formal Documentation	Fill `actual_contributions.csv`; read final PDF; sign `manuscript_approval.md`.	3–4 hrs
Phase 4	Pipeline Integration	Run `study_validated_endpoints.py`; update paper tables; submit to EMSE.	1–2 hrs

Table 1: Structured 5-Phase Implementation Roadmap

3.1 Phase 0: Calibration and Setup (Day 1)

1. **Open the Evidence Index:** Double-click `Evidence_Index.html` in your web browser. This offline-ready index links all retained registry JSONs, commit SHAs, and source-text files.
2. **Open Your Personal Review Sheet:** Open your assigned CSV in a proper editor (LibreOffice Calc, Excel, or VS Code). Ensure UTF-8 encoding is selected and text columns are preserved without scientific formatting.
3. **Calibration Run:** Time yourselves independently on the first 5 rows (`A001` to `A005`). Do not share decisions yet. This establishes your realistic pace (typically 8–15 minutes per endpoint).

3.2 Phase 1: Independent Review Passes (Days 2 to 10)

1. **Batch Schedule:** Break the 120 rows into 12 manageable sessions of 10 rows each (e.g. 1 to 2 sessions per day).
2. **Strict Independence:** Never peek at your coauthor's sheet or compare notes until all 120 rows are completed.
3. **Continuous Local Validation:** At the end of each session, run the automated structural checker:

```
python3 tasks/author_validation_2026-10-28/validate_reviews.py
```

   This checks that your IDs are untouched, all 5 labels are valid, and evidence notes are present.

3.3 Phase 2: Reconciliation and Adjudication (Day 11)

1. **Freeze Initial CSVs:** Once both sheets pass validation, record their SHA-256 checksums. **Never alter initial labels after this point!** Initial disagreement is legitimate scientific data used to calculate Cohen's κ.
2. **Copy to Canonical Location & Detect Disagreements:**

```
cp tasks/author_validation_2026-10-28/reviews/Shahriar_Hasan_independent_labels.csv manuscript_q1
    /review_packets/
cp tasks/author_validation_2026-10-28/reviews/Md_Ashikul_Islam_independent_labels.csv
    manuscript_q1/review_packets/
python3 scripts/study_validated_endpoints.py --label-source human
```

3. **Adjudicate Disagreements:** The script generates `manuscript_q1/review_packets/adjudication.csv` listing only rows where your labels differed. Both authors sit together, review the evidence, and agree on an `adjudicated_label` and documented `reason`.

3.4 Phase 3: Formal Documentation (Day 12)

1. Fill `tasks/author_validation_2026-10-28/templates/actual_contributions.csv`.
2. Read the generated final manuscript PDF in `manuscript/build/main_authors.pdf`.
3. Fill and personally sign `tasks/author_validation_2026-10-28/templates/manuscript_approval.md`.

3.5 Phase 4: Pipeline Integration & Journal Submission (Day 13)

1. Re-run `python3scripts/study_validated_endpoints.py--label-sourcehuman`. This produces verified metrics, stratified population estimates, and Cohen's κ.
2. Rebuild the final paper: tables and text update automatically to reflect adjudicated human results.
3. Submit the updated manuscript package and cover letter via the EMSE Editorial Manager portal.

4 How to Handle These Tasks Practically (Inspection Protocol)

4.1 Standard Operating Procedure for Every Endpoint (8 Steps)

Follow this exact sequence for each audit row:

1. **Identify the Row:** Note the `audit_id` (e.g. `A002`), repository, and pinned `commit_sha` in your CSV.
2. **Find in HTML Index:** Press `Ctrl+F` in `Evidence_Index.html` and type `A002`.
3. **Inspect Repository Mapping (Criterion 1):**
   - Click the retained registry JSON. Note the package name, version, and `repository` URL.
   - Click the pinned GitHub `package.json` (or monorepo subfolder). Verify that `"name"` matches the npm package.
   - If identical, assign `supported`. If mismatched or conflicting, assign `unsupported`. If tree/request failed, assign `unknown`.
4. **Inspect Workflow Presence (Criterion 2):**
   - Check the list of workflow files in `.github/workflows/`.
   - If at least one valid YAML workflow exists at the pinned commit, assign `supported`.
   - If the repository tree was fully inspected and no workflows exist, assign `unsupported`. If incomplete/404, assign `unknown`.
5. **Inspect Install Directive (Criterion 3):**
   - Open the workflow source text (e.g. `test.yml` or `ci.yml`).
   - Search for `run:npmci`, `npminstall`, `pnpmi`, or `yarninstall`.
   - Check working directory: does it apply to this package or the monorepo root?
   - If yes, assign `supported`. If workflows exist but none install dependencies, assign `unsupported`. If unresolvable or reusable workflow inaccessible, assign `unknown`.
6. **Inspect Lockfile Presence (Criterion 4):**
   - Look for `package-lock.json`, `pnpm-lock.yaml`, or `yarn.lock` at root or package level.
   - If present and covering this package, assign `supported`. If proved absent, assign `unsupported`. If uncertain, assign `unknown`.
7. **Inspect Dependency/Version Pair (Criterion 5):**
   - Pick one dependency in `package.json` (e.g. `"typescript":"^5.0.0"`).
   - Open the lockfile and find that package entry. Look for the resolved version (e.g. `5.2.2`).
   - If found and matching, assign `supported`. If lockfile has no dependencies or doesn't resolve it, assign `unsupported`. If lockfile absent or unparseable, assign `unknown`.
8. **Write Evidence Notes:** Enter structured notes in `evidence_notes`. Follow the standard template:

```
Mapping: [package@version -> path/name; registry match];
Workflow: [.github/workflows/ci.yml];
Install: [ci job, step 'Install deps', run: pnpm i];
Lock: [pnpm-lock.yaml covering workspace];
Version: [typescript ^5.0 -> resolved 5.2.2 in lockfile];
Reason: [explanation if unknown or unsupported].
```

4.2 Decision Matrix: Supported vs Unsupported vs Unknown

Criterion	Supported	Unsupported	Unknown
Mapping	Registry URL matches repo AND manifest `package.json` has matching name.	Manifest has conflicting name or repo points elsewhere.	404 error, missing commit, or repo deleted.
Workflow	Readable `.github/workflows/*.yml` file present at pinned commit.	Complete directory tree inspected; zero workflow files exist.	Tree listing failed, truncated, or unreadable.
Install	Active install step found (`npmci`, `pnpmi`, etc.) in relevant path.	Workflows present, but do not execute any install step.	Complex composite/reusable action cannot be resolved.
Lockfile	Relevant lockfile exists at root or in package directory.	Adequately inspected tree shows no lockfile.	Retrieval capped or workspace coverage ambiguous.
Dep/Version	At least 1 dependency in `package.json` matches resolved lockfile entry.	Lockfile lacks the dependency or manifest has zero dependencies.	Lockfile absent, truncated, or format unparseable.

Table 2: Exact Decision Matrix Across the Five Criteria

5 Key Edge Cases and Practical Tips

5.1 How to Handle Monorepos (e.g. A002: arthurfiorette/tinylibs)

- In monorepos, the root `package.json` often names the root workspace rather than the sampled package.
- Look inside the subpackages listed in the HTML index (e.g. `packages/cache-parser/package.json`).
- For lockfiles, root lockfiles (such as `pnpm-lock.yaml` at repository root) usually govern all workspace packages. Document that the root lockfile covers the package workspace.

5.2 Handling Missing Data or 404s

- A 404 response or network error **is NOT unsupported**. It means evidence could not be acquired. Always mark as `unknown`.
- Do not substitute the pinned commit with today's HEAD branch. The research requires inspecting the frozen commit SHA.

5.3 Critical Do's and Don'ts

DO: Use lowercase `supported`, `unsupported`, and `unknown`.

DO: Save frequently and run `python3validate_reviews.py` after every session.

DO: Write specific, informative notes (file paths, step names, package versions).

DON'T: Never peek at or share decisions with your coauthor before both initial passes are finished.

DON'T: Never change your initial CSV during adjudication. Adjudications go into a separate file (`adjudication.csv`).

DON'T: Do not execute, build, or run any repository code. All review is purely static, read-only inspection.

DON'T: Do not leave blank cells. A blank cell means unfinished review, whereas `unknown` means inspected but ambiguous/unavailable.

6 Master Reference of Files and Locations

Summary Reminder

You are now fully equipped to proceed. Start with **Phase 0** today by opening `Evidence_Index.html` and timing your first 5 rows in your respective CSV sheets. Happy reviewing!

File / Directory	Purpose and Usage
`Author_Validation_Guide.pdf`	Printable 18-section comprehensive study protocol.
`Evidence_Index.html`	Offline-ready browser index linking all 120 endpoints, pinned SHAs, and source files.
`Evidence_Packet.md`	Detailed markdown evidence dossier with all raw endpoint records.
`reviews/Md_Ashikul_Islam_independent_labels.csv`	Working review sheet for Md. Ashikul Islam (120 rows).
`reviews/Shahriar_Hasan_independent_labels.csv`	Working review sheet for Shahriar Hasan (120 rows).
`templates/actual_contributions.csv`	Form to describe real author contributions across CRediT roles.
`templates/manuscript_approval.md`	Checklist for formal manuscript review and sign-off.
`templates/review_session_log.csv`	Optional log to record review session times and access issues.
`validate_reviews.py`	Local automated checker for sheet structure and completeness.
`scripts/study_validated_endpoints.py`	Pipeline analysis script computing agreement, kappa, and estimates.

Table 3: Author Validation Directory File Reference